

Министерство образования и науки Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет «Программной инженерии и компьютерной техники»

Сервис-ориентированная архитектура

Лабораторная работа №2

Выполнил:

Григорьев Давид Владимирович и Жук Иван Александрович

Группа: Р3415

Проверил:

Цопа Евгений Алексеевич

Санкт-Петербург, 2026

Содержание

1	Цель работы	3
2	Задание	3
3	Ход работы	3
3.1	Модель данных	3
3.2	Сервис управления организациями	4
3.3	Сервис каталога организаций	5
3.4	Клиентское приложение	6
3.5	Развёртывание	6
3.6	Тестирование	7
3.7	Непрерывная интеграция	8
4	Результат	8
5	Выводы	8
	Приложение А	
		9

1 Цель работы

Реализовать два веб-сервиса на JAX-RS по спецификации лабораторной работы №1, развернуть их в окружении WildFly под HTTPS с самоподписанным сертификатом и подготовить клиентское приложение, использующее API обоих сервисов.

2 Задание

По варианту 67321 нужно реализовать два веб-сервиса и клиентское приложение на основе спецификаций OpenAPI из лабораторной работы №1.

Первый («вызываемый») сервис управляет коллекцией объектов `Organization`: добавляет, получает по идентификатору, обновляет и удаляет элементы, возвращает массив коллекции с фильтрацией, сортировкой и страничным выводом, а также выполняет операции статистики. Он реализован на JAX-RS и развёрнут под управлением WildFly.

Второй сервис расположен на URL `/orgdirectory`, реализован на JAX-RS и вызывает REST API первого сервиса. Он предоставляет две операции: фильтрацию организаций по количеству сотрудников на URL `/filter/employees/{min-employees-count}/{max-employees-count}` и сортировку списка по заданному полю на URL `/order/{param-name}/{desc}`.

Доступ к обоим сервисам должен выполняться по HTTPS с самоподписанным сертификатом; доступ по нешифрованному HTTP должен быть запрещён. Клиентское приложение должно обеспечивать полный набор возможностей API обоих сервисов, включая сортировку, фильтрацию и страничный вывод.

3 Ход работы

3.1 Модель данных

Модель совпадает со спецификацией лабораторной работы №1. Объект `Organization` содержит обязательные поля `name`, `coordinates`, `creationDate`, `type` и `postalAddress`, опциональные `annualTurnover` и `fullName`. Поле `creationDate` и уникальный положительный идентификатор `id` генерируются автоматически. Адрес ограничен улицей длиной не более 158 символов, тип организации принимает одно из значений `COMMERCIAL`, `TRUST`, `PRIVATE_LIMITED_COMPANY`, `OPEN_JOINT_STOCK_COMPANY`.

У организации в спецификации нет поля «количество сотрудников», но операция `/orgdirectory/filter/employees/{min}/{max}` использует это число. Чтобы не изменять модель `Organization`, в БД добавлена сущность `Employee` со связью «многие к одному» на организацию: сотрудник хранит имя, опциональные должность и зарплату и ссылку на организацию. Число сотрудников вычисляется запросом на подресурс `GET /organizations/{id}/employees`.

3.2 Сервис управления организациями

Сервис реализован на JAX-RS (Jakarta EE 10) на Kotlin, упаковывается в `R00T.war` и развёрнут под корневым контекстом. Класс `OrganizationResource` описывает ресурс `/organizations` и его подресурсы:

Ресурс	Метод	Назначение
<code>/organizations</code>	GET	Выборка с фильтрацией, сортировкой и пагинацией
<code>/organizations</code>	POST	Создание организации
<code>/organizations/{id}</code>	GET	Получение организации по идентификатору
<code>/organizations/{id}</code>	PUT	Полное обновление организации
<code>/organizations/{id}</code>	DELETE	Удаление организации
<code>/organizations/{id}/employees</code>	GET, POST	Получение и добавление сотрудников
<code>/organizations/{id}/employees/{employeeId}</code>	DELETE	Удаление сотрудника
<code>/organizations/stats/grouped-by-name</code>	GET	Группировка по name с числом элементов в группе
<code>/organizations/stats/annual-turnover/less-than/{value}</code>	GET	Число организаций с <code>annualTurnover</code> меньше порога
<code>/organizations/stats/annual-turnover/unique</code>	GET	Уникальные значения <code>annualTurnover</code>

Выборка коллекции принимает query-параметры: фильтры по любому полю класса (вложенные поля записываются через точку, например `coordinates.x` и `postalAddress.street`), сортировку по одному или нескольким полям (`sortBy` повторяется, направление задаёт `sortOrder` со значениями `asc/desc`) и пагинацию параметрами `page` и `size`. Сначала применяются фильтры, затем подсчитывается `total`, после чего выбирается страница.

GET `/organizations?type=COMMERCIAL&sortBy=name&sortOrder=asc&page=1&size=10`

Все запросы и ответы передаются в формате `application/xml`; XML формируется и разбирается вручную (`XmlCodec`), без аннотаций JAXB, что сохраняет точное согласованное со спецификацией представление. Код ошибки зависит от причины: 400 Bad Request — нарушение ограничений целостности, 404 Not Found — отсутствующий путь или объект, 415 Unsupported Media Type — тело не в формате XML, 500 — внутренняя ошибка, 503 Service Unavailable — временная недоступность базы данных.

Данные хранятся в PostgreSQL. Схема создаётся миграциями Liquibase при первом запуске; объективное отображение по умолчанию не генерирует схему, существенные таблицы отмечаются как уже применённые миграции. Обращение к БД выполняется через data source `SoaDS`, настроенный в `WildFly`.

3.3 Сервис каталога организаций

Второй сервис упаковывается в `orgdirectory.war`, развёрнут по контексту `/orgdirectory` и вызывает API первого сервиса по HTTPS. Класс `DirectoryResource` объявляет две операции:

Путь	Метод	Назначение
<code>/filter/employees/{min}/{max}</code>	GET	Фильтрация по диапазону числа сотрудников, границы включены
<code>/order/{param-name}/{desc}</code>	GET	Сортировка по заданному полю (<code>false</code> — по возрастанию, <code>true</code> — по убыванию)

Обе операции вызывают `DirectoryService`. Фильтрация запрашивает все организации постранично по 100 элементов через `OrganizationApiRepository`, для каждой считает число сотрудников запросом `GET /organizations/{id}/employees` и оставляет организации, попадающие в диапазон. Сортировка передаёт выбранное поле и направление в первый сервис; допустимые поля перечислены в наборе `id`, `name`, `coordinates.x`, `coordinates.y`, `creationDate`, `annualTurnover`, `fullName`, `type`, `postalAddress.street`.

Результат — XML `<organizations>` с атрибутом `total` и вложенными `<organization>`, полученными от первого сервиса без изменения. При некорректных параметрах (минимум больше максимума, недопустимое поле или направление) возвращается 400 Bad Request. Ошибки взаимодействия транслируются в 502 Bad Gateway («Organization Service вернул ошибочный ответ»),

а превышение таймаута ожидания ответа (12 секунд на запрос, 4 секунды на установление соединения) — в 504 Gateway Timeout.

3.4 Клиентское приложение

Клиентское приложение — интерактивная веб-документация Swagger UI, собранная в `ui.war` и доступная по URL `/ui/`. Она подгружает обе «живые» спецификации OpenAPI (Organization Service и OrgDirectory Service), которые WildFly отдаёт по адресам `/openapi/organizations` и `/openapi/orgdirectory`. Ввод запроса, включая параметры фильтрации, сортировки и пагинации, выполняется через кнопку «Try it out»; ответ показывается в человекочитаемом виде с кодом состояния, что также даёт пользователю информацию об ошибках сервисов.

Спецификации используют относительные server URL, поэтому «Try it out» обращается к текущему хосту и порту, что работает в том числе при пробросе порта по SSH-туннелю. В `<noscript>` и при отключённой сети UI показывает подсказку.

3.5 Развёртывание

Оба сервиса и клиент развёрнуты в окружении WildFly 35. Для локальной проверки используется стек `wildfly/compose.yaml`: контейнер `postgres:16-alpine` (порт наружу 15432) и контейнер WildFly на базе `quay.io/wildfly/wildfly:35.0.1.Final-jdk21`, собранный `wildfly/Dockerfile`. HTTPS-порт контейнера 61811 пробрасывается на `127.0.0.1:62811`, чтобы к тестовой среде не было доступа извне.

Компонент	Назначение	Порт
WildFly	Сервера приложений для обоих JAX-RS-сервисов и Swagger UI	61811 (наружу 62811)
PostgreSQL	Хранилище, схема <code>s389491</code> , миграции Liquibase	5432 (наружу 15432)
Management	Администрирование WildFly, только localhost	61812

TLS настроен на один HTTPS-листелнер с самоподписанным сертификатом, сгенерированным задачей Gradle `generateTls` (`tls/server.p12`, `truststore` `tls/truststore.p12`, SAN включает `localhost`, `helios`, `se.ifmo.ru`, `127.0.0.1`). Обычный HTTP-листелнер отключён — доступ только по HTTPS; plaintext-

запрос на порт получает код ошибки, а management-порты слушают только на localhost. Пароль БД хранится в приватном файле .env (права 600) и ссылается из конфигурации data source через переменные окружения.

Скрипт deploy-wildfly.sh собирает WAR-файлы и материал TLS, передаёт их на сервер helios по scp в каталог ~/soa-lab2 и запускает setup-wildfly.sh, который устанавливает WildFly (при отсутствии), разворачивает приложения, применяет настройки data source и TLS и стартует сервер. Адреса после развёртывания: UI <https://se.ifmo.ru:61811/ui/>, организации <https://se.ifmo.ru:61811/organizations>, каталог <https://se.ifmo.ru:61811/orgdirectory/order/name/false>.

Оба JAX-RS-приложения и Swagger UI на текущий момент развёрнуты в одном общем экземпляре WildFly; требование задания о размещении второго сервиса в отдельном экземпляре реализуется при развёртывании на helios отдельным WAR-модулем (R00T.war и orgdirectory.war) и настраивается независимыми data source.

3.6 Тестирование

Сборка сопровождается тестами на нескольких уровнях.

E2E-набор (WildFlyStackE2ETest) запускается задачей ./gradlew e2eTest, которая поднимает стек из compose.yaml, выполняет тесты против работающей HTTPS-установки и останавливает стек. Тесты проверяют полный CRUD со схемой s389491 и автоматически генерируемыми полями, фильтрацию/сортировку/пагинацию, сотрудников и агрегатные операции, работу каталога через реальный upstream по TLS, статусы и сообщения валидационных ошибок, отдачу Swagger UI и спецификаций OpenAPI вместе с CORS, а также запрет доступа по plaintext HTTP.

Проверка примеров OpenAPI (./gradlew :e2e-tests:openapiExamples) выполняет все явные XML-примеры обеих спецификаций против одноразового стека и сравнивает ответы канонически (без учёта порядка атрибутов и несущественных пробелов). Дополнительно проверяется, что отдаваемые WildFly «живые» спецификации сохраняют XML-контракты запросов из исходных YAML. Отказоустойчивость проверяется контролируемо: переименованием таблицы (ответ 500), остановкой контейнера БД (ответ 503) и подменой адреса upstream на тестовый сервер, который отвечает с задержкой (ответ 504) или ошибкой (ответ 502).

Независимые проверки (построение WAR, подготовка модуля драйвера JDBC, генерация TLS, сборка Swagger UI) выполняются отдельными задачами Gradle, что делает сборку воспроизводимой без ручных шагов.

3.7 Непрерывная интеграция

Для репозитория настроен GitHub Actions: рабочий процесс `ci.yml` запускается на push в `main` и на pull request. Он собирает проект JDK 17, строит оба WAR и Swagger UI, поднимает Docker-стек, выполняет e2e-набор, а затем прогон OpenAPI-примеров. Рабочий процесс имеет ограничение по времени и отменяет предыдущий запуск при новом пуше.

4 Результат

Реализованы два JAX-RS-сервиса: управление коллекцией организаций (CRUD, фильтрация/сортировка/пагинация, три операции статистики, сотрудники) и каталог `/orgdirectory` (фильтрация по числу сотрудников и сортировка по полю). Оба сервиса отвечают в XML в строгом соответствии со спецификациями лабораторной работы №1, общаются между собой по HTTPS, работают только по защищённому протоколу с самоподписанным сертификатом. Клиентское приложение Swagger UI даёт полный доступ к API обоих сервисов. Функциональность покрыта e2e-тестами, прогоном всех OpenAPI-примеров и включена в CI.

5 Выводы

В ходе работы освоено развёртывание JAX-RS-приложений в WildFly: настройка HTTPS-only листенера с самоподписанным сертификатом, data source к PostgreSQL и модуль JDBC-драйвера, миграции Liquibase. Второй сервис показал интеграцию по HTTPS между сервисами: он переиспользует страничную выборку первого сервиса, транслирует его ошибки в 502/504 и выполняет требования спецификации без изменения модели `Organization` (число сотрудников считается по связанным сотрудникам). Автоматические проверки на нескольких уровнях и CI позволяют подтверждать соответствие реализации спецификации при каждом изменении.

Приложение А

Исходный код сервисов и клиентского приложения

Исходный код обоих сервисов, клиентского приложения, тестов и скриптов развёртывания размещён в репозитории проекта: <https://github.com/paryashie-yasheri/soa/>